Listing 12.9 shows the `RecipeService.UpdateRecipe` method, which updates the `Recipe` entity. It performs the three steps we defined previously to read, modify, and save the entity. I've extracted the code to update the recipe with the new values to a helper method for clarity.

Listing 12.9 Updating an existing entity with EF Core in `RecipeService`

```
public async Task UpdateRecipe(UpdateRecipeCommand cmd)
{
    var recipe = await _context.Recipes.FindAsync(cmd.Id);          ❶
    if(recipe is null) {                                            ❷
        throw new Exception("Unable to find the recipe");           ❷
    }                                                               ❷
    UpdateRecipe(recipe, cmd);                                      ❸
    await _context.SaveChangesAsync();                              ❹
}

static void UpdateRecipe(Recipe recipe, UpdateRecipeCommand cmd)    ❺
{                                                                   ❺
    recipe.Name = cmd.Name;                                         ❺
    recipe.TimeToCook =                                             ❺
        new TimeSpan(cmd.TimeToCookHrs, cmd.TimeToCookMins, 0);     ❺
    recipe.Method = cmd.Method;                                     ❺
    recipe.IsVegetarian = cmd.IsVegetarian;                         ❺
    recipe.IsVegan = cmd.IsVegan;                                   ❺
}                                                                   ❺
```

❶ Find is exposed directly by Recipes and simplifies reading an entity by id.

❷ If an invalid id is provided, recipe will be null.

❸ Sets the new values on the Recipe entity

❹ Executes the SQL to save the changes to the database

❺ A helper method for setting the new properties on the Recipe entity

In this example I read the `Recipe` entity using the `FindAsync(id)` method exposed by `DbSet`. This simple helper method loads an entity by its ID—in this case, `RecipeId`. I could have written a similar query with LINQ: